

Algoritmos sobre grafos

Representaciones, ordenamiento topológico, BFS y DFS

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2026

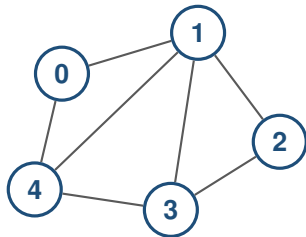
Agenda de hoy

- 1 Representación de grafos.
- 2 Ordenamiento topológico
- 3 Búsqueda a lo ancho (BFS).
- 4 Búsqueda en profundidad (DFS).
- 5 Detección de aristas de corte.

Basado en Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, cap. 20.

El grafo y su representación son objetos diferentes

Objeto matemático



$$G = (V, E)$$

Estructura de datos

- Debe almacenar los vértices y las aristas.
- Debe permitir las operaciones que necesita el algoritmo.
- Distintas representaciones inducen distintos costos.

No hay una representación universalmente mejor.

Representamos el grafo mediante sus vecindarios

En la práctica suponemos

$$V = \{0, \dots, n-1\}.$$

Para cada vértice v ,

$$N(v) = \{w \in V : vw \in E\}.$$

Una representación computacional implementa la función

$$v \mapsto N(v).$$

Operaciones que nos interesan

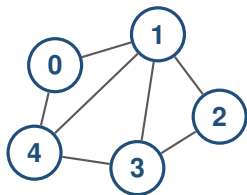
consultar si $vw \in E$

recorrer $N(v)$

insertar o remover aristas

insertar o remover vértices

Un mismo grafo admite distintas representaciones

Grafo**Listas**

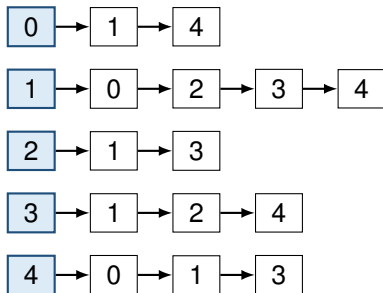
$0 \mapsto 1, 4$
 $1 \mapsto 0, 2, 3, 4$
 $2 \mapsto 1, 3$
 $3 \mapsto 1, 2, 4$
 $4 \mapsto 0, 1, 3$

Matriz

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

La elección no cambia el grafo; cambia cómo accedemos a su información.

Las listas almacenan solamente los vecinos existentes



$$\text{Adj}[v] = N(v).$$

- Hay una lista enlazada por vértice.
- En un digrafo se guardan, usualmente, los vecinos de salida:

$$N^+(v) = \{w : v \rightarrow w \in E\}.$$

- El orden de los vecinos no está determinado por el grafo.

El espacio de las listas es lineal en el tamaño del grafo

Digrafos

$$\sum_{v \in V} |\text{Adj}[v]| = m.$$

Cada arco $v \rightarrow w$ aparece una sola vez, en la lista de v .

Grafos no dirigidos

$$\sum_{v \in V} |\text{Adj}[v]| = 2m.$$

Cada arista vw aparece en las listas de v y de w .

n encabezados
uno por vértice



m o $2m$ entradas
una o dos por arista

Espacio total: $\Theta(n + m)$.

Los costos dependen de cómo implementamos cada lista

Supongamos listas no ordenadas, acceso directo a $\text{Adj}[v]$ e inserción al frente.

Consultas

recorrer $N(v) : O(d(v))$,
decidir si $vw \in E : O(d(v))$.

Para decidir adyacencia debemos buscar w dentro de la lista de v .

Actualizaciones

insertar $vw : O(1)$,
remover $vw : O(d(v) + d(w))$,
remover $v : O(n + m)$ (peor caso).

La matriz reserva una posición para cada par de vértices

$$A[v, w] = \begin{cases} 1, & \text{si } vw \in E, \\ 0, & \text{si } vw \notin E. \end{cases}$$

- Espacio: $\Theta(n^2)$.
- Consultar $vw \in E$: $O(1)$.
- Recorrer $N(v)$: $O(n)$.
- Si G es no dirigido, $A = A^T$.

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

La fila v codifica $N(v)$.

La operación dominante orienta la elección

	Listas	Matriz
Espacio	$\Theta(n + m)$	$\Theta(n^2)$
Consultar $vw \in E$	$O(d(v))$	$O(1)$
Recorrer $N(v)$	$O(d(v))$	$O(n)$
Recorrer todas las aristas	$O(n + m)$	$O(n^2)$

Grafo con pocas aristas

suelen convenir las listas porque
 $n + m \ll n^2$ si $m \sim n$.

Grafo denso

puede convenir la matriz.

Los pesos se almacenan junto con las aristas

Listas de adyacencia

Cada entrada guarda el vecino y el peso:

$$\text{Adj}[v] \ni (w, p(v, w)).$$

Por ejemplo,

$$\text{Adj}[0] = [(1, 7), (4, 3)].$$

Matriz de adyacencia

$$A[v, w] = \begin{cases} p(v, w), & vw \in E, \\ \text{NIL}, & vw \notin E. \end{cases}$$

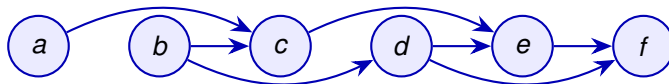
Cuidado

No conviene usar 0 para indicar ausencia si una arista puede tener peso 0.

La estructura básica no cambia: cambia la información asociada a cada arista.

Un orden topológico respeta todas las aristas

Sea $D = (V, E)$ un digrafo. Un **ordenamiento topológico** de D es un orden lineal $v_1, v_2, \dots, v_n$ de todos sus vértices tal que $v_i \rightarrow v_j \in E \implies i < j$.



todas las flechas avanzan en el orden

Lema

Todo digrafo acíclico (DAG) tiene un vértice v tal que $d^{\ell}(v) = 0$.

Teorema

Un digrafo admite un ordenamiento topológico si y solo si es acíclico .

Vértices con grado de entrada igual a cero

En un DAG siempre existe algún vértice de grado de entrada cero. Cualquiera de ellos puede ocupar la próxima posición del orden.

Algoritmo recursivo

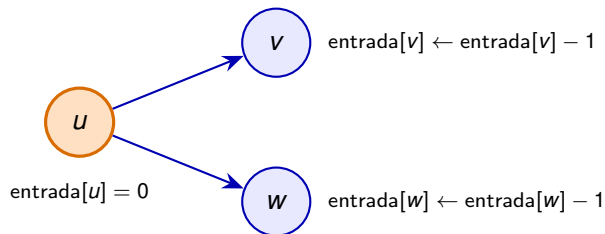
- 1 Buscar un vértice u tal que $d_D^-(u) = 0$ y agregarlo al final del orden.
- 2 Eliminar u de D .
- 3 Aplicar recursivamente el algoritmo al digrafo $D - u$.
- 4 Si el digrafo restante no es vacío y no tiene ningún vértice de grado de entrada cero, entonces contiene un ciclo dirigido.

Complejidad

Con listas de adyacencia, buscar y eliminar un vértice cuesta $O(n + m)$. Como se realizan hasta n pasos, esta implementación cuesta $O(n(n + m))$.

El próximo vértice debe tener grado de entrada cero

En un DAG siempre existe algún vértice de grado de entrada 0. Cualquiera de ellos puede ocupar la próxima posición del orden.



- 1 Agregar u al final del orden.
- 2 Eliminar conceptualmente a u .
- 3 Disminuir el grado de entrada de cada vecino saliente.
- 4 Encolar los vecinos cuyo grado pasa a ser 0.

Solo se modifican los grados de entrada de los vértices $v \in N^+(u)$.

Una cola mantiene los vértices disponibles

ORDEN-TOPOLÓGICO(D)

Inicialización

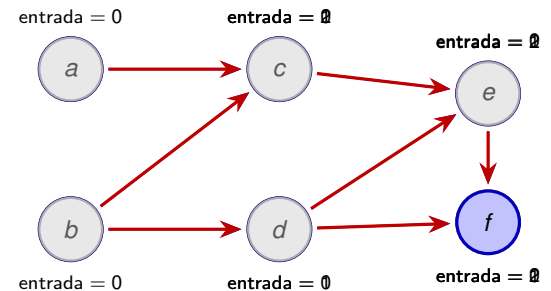
```
1  $n \leftarrow |V(D)|$       $L \leftarrow \langle \rangle$   
2 computar entrada[ $v$ ] =  $d^-(v)$  para todo  $v \in V(D)$   
3  $Q \leftarrow$  cola vacía  
4 para todo  $v \in V(D)$  hacer  
5     si entrada[ $v$ ] = 0 entonces  
6         ENCOLAR( $Q, v$ )
```

Construcción del orden

```
7 mientras  $Q \neq \emptyset$  hacer  
8      $u \leftarrow$  DESENCOLAR( $Q$ )  
9     agregar  $u$  al final de  $L$   
10    para todo  $v \in N^+(u)$  hacer  
11      entrada[ $v$ ]  $\leftarrow$  entrada[ $v$ ] - 1  
12      si entrada[ $v$ ] = 0 entonces  
13         ENCOLAR( $Q, v$ )  
14 si  $|L| < n$  entonces  
15     devolver " $D$  tiene un ciclo"  
16 devolver  $L$ 
```

Cada vértice se encola una sola vez y cada arista se procesa una sola vez.

Ejemplo: la cola se actualiza al eliminar cada vértice



■ en la cola

■ elegido

■ procesado

Inicialización

$$Q = \langle a, b \rangle, \quad L = \langle \rangle.$$

Los únicos vértices con grado de entrada 0 son *a* y *b*.

Procesamos *a*

$$Q = \langle b \rangle, \quad L = \langle a \rangle.$$

Al eliminar $a \rightarrow c$, el grado de entrada de *c* baja de 2 a 1.

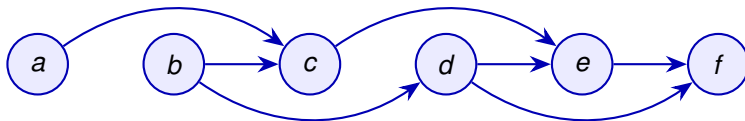
Procesamos *b*

$$Q = \langle c, d \rangle, \quad L = \langle a, b \rangle.$$

El resultado coloca el origen antes que el destino

La ejecución anterior devuelve

$$L = \langle a, b, c, d, e, f \rangle.$$



todas las aristas apuntan hacia la derecha

Costo

Con listas de adyacencia,

$$O(|V(D)| + |E(D)|).$$

Si la cola se vacía antes

Si $|L| < |V(D)|$, el subdigrafo inducido por los vértices restantes contiene un ciclo; no existe orden topológico.

Recorrer un grafo

Muchos algoritmos sobre grafos siguen un mismo esquema para recorrer los vértices alcanzables desde un vértice inicial s .

- 1 Descubrir el vértice inicial s .
- 2 Mantener una colección de vértices descubiertos que todavía no fueron procesados.
- 3 Elegir uno de esos vértices u , procesarlo y examinar sus vecinos.
- 4 Incorporar a la colección los vecinos de u que todavía no fueron descubiertos.

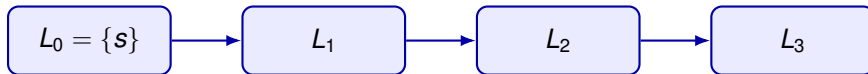
Estructura	Orden de procesamiento	Recorrido
Cola	primero el más antiguo	BFS
Pila	primero el más reciente	DFS

BFS explora por distancia creciente desde una fuente

Sea $G = (V, E)$ un grafo o digrafo **sin pesos** y sea $s \in V$ un vértice fuente. La búsqueda a lo ancho (BFS) explora primero los vértices más cercanos a s .

$$L_i = \{v \in V : \delta(s, v) = i\},$$

donde $\delta(s, v)$ es la longitud de un camino mínimo de s a v , y vale ∞ si v no es alcanzable desde s .



BFS calcula:

- $d[v]$: la distancia desde s hasta v ;
- $\pi[v]$: el predecesor de v en un árbol de caminos mínimos.

Los colores describen el estado de cada vértice

Durante la ejecución, cada vértice se encuentra en uno de tres estados.

Blanco: todavía no fue descubierto

Su distancia es $d[v] = \infty$ y su predecesor es $\pi[v] = \text{NIL}$.

Gris: fue descubierto, pero no terminó de procesarse

El vértice está en la cola. Es parte de la frontera entre los vértices descubiertos y los que todavía no fueron descubiertos.

Negro: terminó de procesarse

Toda su lista de adyacencia ya fue examinada.

Al comienzo de cada iteración, la cola contiene exactamente los vértices grises.

BFS utiliza una cola para administrar la frontera

Función $\text{BFS}(G, s)$

para cada $u \in V(G) \setminus \{s\}$ **hacer**

$\text{color}[u] \leftarrow \text{blanco}$

$d[u] \leftarrow \infty, \quad \pi[u] \leftarrow \text{NIL}$

$\text{color}[s] \leftarrow \text{gris}$

$d[s] \leftarrow 0, \quad \pi[s] \leftarrow \text{NIL}$

$Q \leftarrow \text{cola vacía}; \quad \text{ENCOLAR}(Q, s)$

mientras $Q \neq \emptyset$ **hacer**

$u \leftarrow \text{DESENCOLAR}(Q)$

para cada $v \in \text{Adj}[u]$ **hacer**

si $\text{color}[v] = \text{blanco}$ **entonces**

$\text{color}[v] \leftarrow \text{gris}$

$d[v] \leftarrow d[u] + 1, \quad \pi[v] \leftarrow u$

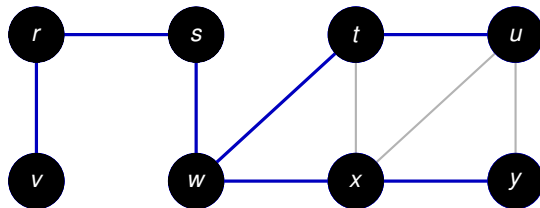
$\text{ENCOLAR}(Q, v)$

$\text{color}[u] \leftarrow \text{negro}$

devolver d, π

Ejemplo: la cola obliga a procesar el grafo por capas

Suponemos que las listas de adyacencia están en orden alfabético.

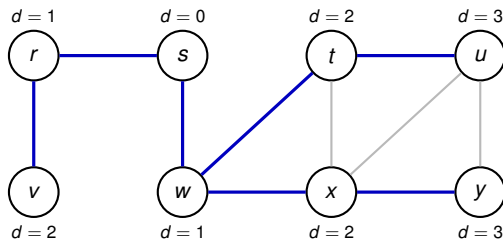


Inicialización: $Q = \langle s \rangle$.

Se procesa s : se descubren r, w y $Q = \langle r, w \rangle$.

Se procesa r : se descubre v y $Q = \langle w, v \rangle$.

Al terminar, las capas coinciden con las distancias



i	$L_i = \{v : d[v] = i\}$	predecesores
0	$\{s\}$	$\pi[s] = \text{NIL}$
1	$\{r, w\}$	$\pi[r] = \pi[w] = s$
2	$\{v, t, x\}$	$\pi[v] = r, \pi[t] = \pi[x] = w$
3	$\{u, y\}$	$\pi[u] = t, \pi[y] = x$

El orden de las listas puede cambiar el árbol, pero no las distancias.

Cada valor $d[v]$ es la longitud de un camino encontrado

Lema

Al terminar BFS, para todo $v \in V(G)$,

$$d[v] \geq \delta(s, v).$$

Demostración. El único valor finito inicial es $d[s] = 0$. Cuando v es descubierto desde u , el algoritmo asigna

$$\pi[v] = u, \quad d[v] = d[u] + 1.$$

Sale por inducción sobre el número de descubrimientos (número de llamadas a ENCOLAR), los predecesores determinan un camino de s a v de longitud $d[v]$. Como $\delta(s, v)$ es la longitud mínima de un camino de s a v ,

$$\delta(s, v) \leq d[v].$$

Falta probar que BFS no encuentra un camino más largo que el mínimo.

La cola contiene vértices de a lo sumo dos capas

Lema de la cola

Si $Q = \langle v_1, v_2, \dots, v_r \rangle$, entonces $d[v_1] \leq d[v_2] \leq \dots \leq d[v_r] \leq d[v_1] + 1$.

Idea de la demostración. Se usa inducción sobre las operaciones realizadas sobre Q .

- Al comienzo, $Q = \langle s \rangle$, y la propiedad es inmediata.
- Al desencolar v_1 , el nuevo primer elemento v_2 satisface $d[v_2] \geq d[v_1]$; las desigualdades se conservan.
- Si v se descubre al procesar u , entonces $d[v] = d[u] + 1$. Al agregarlo al final, queda detrás de vértices con distancia $d[u]$ o $d[u] + 1$.

En consecuencia, los vértices se encolan y se procesan en orden no decreciente de d .

BFS calcula las distancias mínimas desde s

Teorema

Para todo $v \in V(G)$, al terminar BFS, $d[v] = \delta(s, v)$. Además, para todo vértice $v \neq s$ alcanzable desde s , un camino mínimo de s a v se obtiene tomando un camino mínimo de s a $\pi[v]$ y agregando la arista

$$(\pi[v], v).$$

Idea de la demostración. Supongamos que la igualdad falla y elijamos un vértice v con $\delta(s, v)$ mínima entre los que tienen un valor incorrecto.

El preprocesamiento toma $O(n)$. Con listas de adyacencia, cada vértice se encola a lo sumo una vez y cada lista se examina a lo sumo una vez. Así que la complejidad es $O(n) + O(\sum_{v \in V(G)} d(v)) = O(n + 2m) = O(n + m)$.

Árbol BFS y reconstrucción de caminos mínimos

Subgrafo de predecesores

Después de ejecutar $\text{BFS}(G, s)$, definimos $G_\pi = (V_\pi, E_\pi)$, donde

$$V_\pi = \{s\} \cup \{v \in V : \pi[v] \neq \text{NIL}\}, \quad E_\pi = \{(\pi[v], v) : v \in V_\pi \setminus \{s\}\}.$$

Un **árbol BFS con raíz** s contiene exactamente los vértices alcanzables desde s , y el camino del árbol de s a cada uno de ellos es un camino mínimo en G .

Lema

El subgrafo de predecesores G_π es un árbol BFS con raíz s . Para todo vértice v alcanzable desde s , el único camino simple de s a v en G_π es un camino mínimo en G , y su longitud es $d[v] = \delta(s, v)$.

PRINT-PATH(G, s, v)

si $v = s$ **entonces**

 IMPRIMIR(s)

sino, si $\pi[v] = \text{NIL}$ **entonces**

 IMPRIMIR("no existe camino")

sino

 PRINT-PATH($G, s, \pi[v]$)

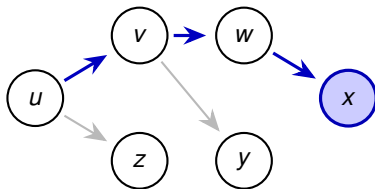
 IMPRIMIR(v)

¿Descansamos 10 minutos?



DFS avanza mientras encuentra vértices no descubiertos

La búsqueda en profundidad (DFS) explora las aristas que salen del vértice **descubierto más recientemente** que todavía tiene aristas sin examinar.



Si puede avanzar: visita recursivamente un vecino blanco.

Si no puede avanzar: retrocede al vértice desde el cual llegó.

Cuando termina de explorar lo alcanzable desde una fuente, el algoritmo elige otro vértice no descubierto. Por eso, en general, produce un **bosque** y no un único árbol.

Los predecesores forman un bosque DFS

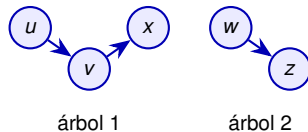
Cuando DFS descubre a v al examinar una arista (u, v) , asigna

$$\pi[v] \leftarrow u.$$

El **subgrafo de predecesores** es

$$G_\pi = (V, E_\pi), \quad E_\pi = \{(\pi[v], v) : v \in V \text{ y } \pi[v] \neq \text{NIL}\}.$$

- G_π es el **bosque DFS**.
- Cada llamada inicial a $\text{DFS-VISIT}(G, u)$ crea una raíz.
- Las aristas de E_π son las **aristas del árbol**.



A diferencia de BFS, DFS no está asociada a una única fuente: el bucle exterior garantiza que todos los vértices pertenezcan a algún árbol.

Los colores indican el estado de la llamada recursiva

Blanco: todavía no fue descubierto

No se inició ninguna llamada a DFS-VISIT para el vértice.

Gris: fue descubierto, pero aún no terminó

La llamada recursiva está activa. Todavía puede quedar alguna arista de su lista de adyacencia sin examinar.

Negro: terminó de procesarse

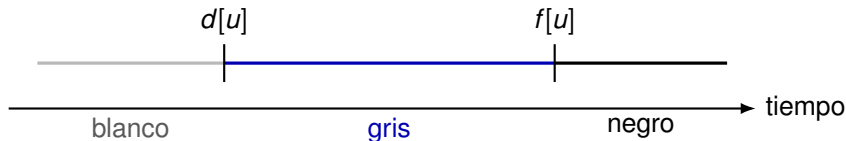
Ya se examinó completamente su lista de adyacencia.

Los vértices grises son exactamente los vértices de la pila de llamadas recursivas.

Tiempos de descubrimiento y de finalización

DFS mantiene un contador global *tiempo*. Para cada vértice u , registra:

- $d[u]$: instante en que u se descubre y pasa a gris;
- $f[u]$: instante en que termina de examinarse y pasa a negro.



Como hay un descubrimiento y una finalización por vértice, los tiempos son enteros de 1 a $2|V|$. En particular,

$$1 \leq d[u] < f[u] \leq 2|V|.$$

DFS inicia una visita desde cada vértice que sigue blanco

Procedimiento DFS(G)

para cada $u \in V(G)$ **hacer**

$\text{color}[u] \leftarrow \text{blanco}$

$\pi[u] \leftarrow \text{NIL}$

$\text{tiempo} \leftarrow 0$

para cada $u \in V(G)$ **hacer**

si $\text{color}[u] = \text{blanco}$ **entonces**

 DFS-VISIT(G, u)

La primera iteración inicializa todos los vértices. La segunda recorre $V(G)$: cada llamada realizada allí crea un nuevo árbol del bosque DFS.

DFS-VISIT profundiza y luego retrocede

Procedimiento DFS-VISIT(G, u)

$tiempo \leftarrow tiempo + 1$

$d[u] \leftarrow tiempo$

$color[u] \leftarrow \text{gris}$

para cada $v \in \text{Adj}[u]$ **hacer**

si $color[v] = \text{blanco}$ **entonces**

$\pi[v] \leftarrow u$

 DFS-VISIT(G, v)

$tiempo \leftarrow tiempo + 1$

$f[u] \leftarrow tiempo$

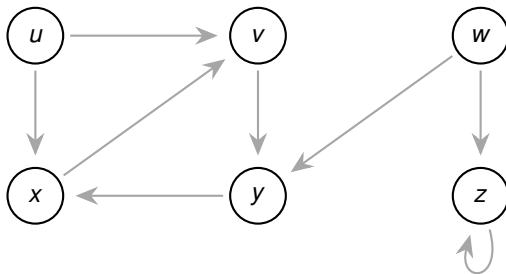
$color[u] \leftarrow \text{negro}$

La llamada de u queda suspendida mientras se explora recursivamente un vecino blanco. Solo finaliza cuando ya se examinaron todas las aristas que salen de u .

Ejemplo dirigido: fijamos el orden de exploración

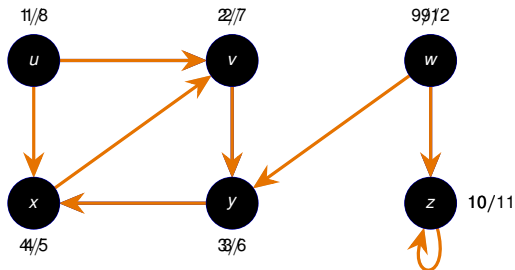
El bucle exterior considera u, v, w, x, y, z , en ese orden, y las listas de adyacencia son:

q	$\text{Adj}[q]$	q	$\text{Adj}[q]$	q	$\text{Adj}[q]$
u	$\langle v, x \rangle$	v	$\langle y \rangle$	w	$\langle y, z \rangle$
x	$\langle v \rangle$	y	$\langle x \rangle$	z	$\langle z \rangle$



Estos órdenes determinan una corrida concreta; otros órdenes pueden producir otro bosque y otros tiempos.

Corrida paso a paso: DFS profundiza antes de retroceder



$t = 1$: el bucle exterior encuentra u blanco y lo descubre.

Pila: $\langle u \rangle$.

$t = 2$: se examina (u, v) ; como v está blanco, $\pi[v] = u$ y se lo descubre.

Pila: $\langle u, v \rangle$.

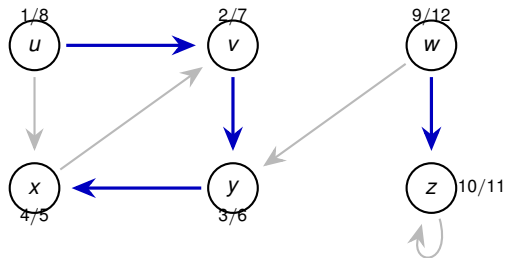
$t = 3$: se examina (v, y) ; como y está blanco, $\pi[y] = v$.

Pila: $\langle u, v, y \rangle$.

$t = 4$: se examina (y, x) ; como x está blanco, $\pi[x] = y$.

Pila: $\langle u, v, y, x \rangle$.

La corrida produce dos árboles y doce marcas temporales



q	$d[q]$	$f[q]$	$\pi[q]$
u	1	8	NIL
v	2	7	u
y	3	6	v
x	4	5	y
w	9	12	NIL
z	10	11	w

Azul: aristas del bosque DFS.

Orden de descubrimiento: u, v, y, x, w, z .

Orden de finalización: x, y, v, u, z, w .

El orden puede cambiar el bosque

La salida concreta de DFS puede depender de dos decisiones:

- el orden en que el bucle exterior considera los vértices;
- el orden de los vértices en cada lista de adyacencia.

Esos órdenes pueden modificar las raíces, las aristas del bosque y los tiempos d y f .

Tiempo de ejecución con listas de adyacencia

La inicialización y el bucle exterior cuestan $\Theta(|V|)$.

DFS-VISIT se llama exactamente una vez por vértice y, en total, el bucle recorre todas las aristas una vez

$$\sum_{u \in V} |\text{Adj}[u]| = \Theta(|E|).$$

Por lo tanto, el tiempo total es

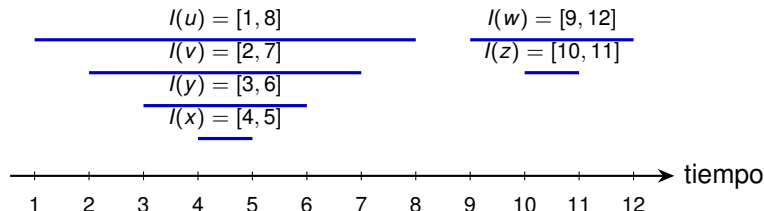
$$\boxed{\Theta(|V| + |E|)}.$$

Los tiempos de DFS tienen una estructura de paréntesis

El intervalo

$$I(u) = [d[u], f[u]]$$

representa el período durante el cual la llamada de u está activa. En la corrida anterior, los intervalos de cada árbol se anidan:



Si al descubrir u escribimos $(u$ y al finalizarlo escribimos $)$, obtenemos una expresión de paréntesis bien formada: una llamada recursiva debe terminar antes de que termine la llamada que la creó.

Dos intervalos de DFS se anidan o son disjuntos

Teorema de los paréntesis

Para cualesquiera $u, v \in V$, ocurre exactamente una de las siguientes posibilidades:

- 1 $I(u) \cap I(v) = \emptyset$, y ninguno es descendiente del otro;
- 2 $I(u) \subset I(v)$, y u es descendiente de v ;
- 3 $I(v) \subset I(u)$, y v es descendiente de u .

Idea de la demostración.

Supongamos $d[u] < d[v]$. Si v se descubre antes de que termine u , entonces u está gris: la exploración de v se completa antes de regresar a u , y $I(v) \subset I(u)$. Si u ya había terminado, entonces $f[u] < d[v]$, y los intervalos son disjuntos.

Dos intervalos nunca se superponen parcialmente.

Los tiempos permiten reconocer descendientes

Corolario

Un vértice v es descendiente propio de u en el bosque DFS si y solo si

$$d[u] < d[v] < f[v] < f[u].$$

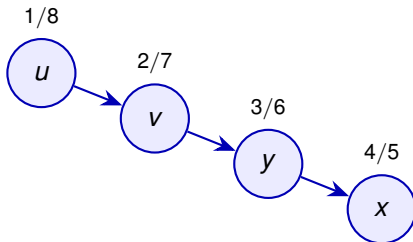
Por ejemplo,

$$1 < 4 < 5 < 8,$$

luego x es descendiente de u .

Idea de la demostración.

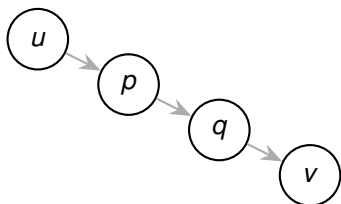
Es la traducción directa del caso de intervalos anidados del teorema de los paréntesis.



Un camino blanco caracteriza a los descendientes

Teorema del camino blanco

En el bosque DFS, v es descendiente de u si y solo si, en el instante $d[u]$, existe un camino de u a v formado enteramente por vértices blancos.



camino completamente blanco
al descubrir u

Idea de la demostración.

- $\Rightarrow$ El camino de u a cualquiera de sus descendientes en el árbol DFS todavía está blanco cuando se descubre u .
- $\Leftarrow$ Si DFS no incorporara todo el camino blanco, tomemos el primer vértice que queda afuera. Su predecesor sí es descendiente de u y, al examinar la arista que los une, encontraría blanco al siguiente vértice: contradicción.

Las aristas de un digrafo se dividen en cuatro tipos

La clasificación se realiza con respecto al bosque DFS obtenido.

Árbol. La arista (u, v) descubre por primera vez a v , por lo que

$$\pi[v] = u.$$

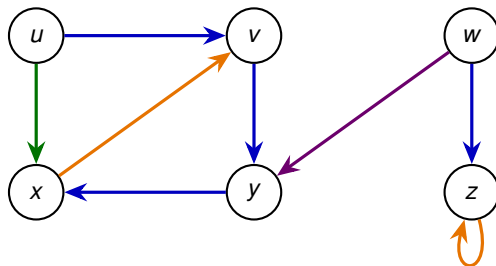
Retroceso. La arista (u, v) va desde u hacia uno de sus ancestros v . Los bucles se consideran aristas de retroceso.

Avance. La arista (u, v) no pertenece al bosque y va hacia un descendiente propio de u .

Cruce. Es cualquier otra arista: sus extremos son incomparables en el árbol DFS o pertenecen a árboles distintos.

La clasificación depende del bosque DFS y, por lo tanto, puede cambiar cuando cambia el orden de exploración.

La corrida anterior contiene los cuatro tipos de arista



→ tree edge

→ back edge

→ forward edge

→ cross edge

$u \rightarrow x$ es una forward edge, $x \rightarrow v$ es una back edge, $w \rightarrow y$ es una cross edge.

El color del destino clasifica la arista al explorarla

Cuando DFS examina una arista (u, v) , el vértice u está gris.

color[v]	tipo de (u, v)	razón
blanco	árbol	v se descubre mediante (u, v)
gris	retroceso	v es un ancestro activo de u
negro	avance o cruce	v ya terminó de procesarse

Idea clave.

Los vértices grises forman una cadena de ancestros: son exactamente las llamadas activas de la pila. Por eso, una arista hacia un vértice gris necesariamente vuelve hacia un ancestro.

Si v está negro, los tiempos distinguen los dos casos:

$$d[u] < d[v] \implies \text{avance}, \quad d[v] < d[u] \implies \text{cruce}.$$

En grafos no dirigidos no hay aristas de avance ni de cruce

Teorema

En una búsqueda en profundidad de un grafo no dirigido G , toda arista es una arista de árbol o una arista de retroceso.

Idea de la demostración.

Sea $\{u, v\} \in E$ y supongamos $d[u] < d[v]$.

- Si la arista se examina primero desde u , entonces v todavía está blanco y la arista se incorpora al árbol.
- Si se examina primero desde v , entonces u todavía está gris y es un ancestro de v ; la arista es de retroceso.

Estas dos posibilidades agotan los casos.

Para hallar los puentes alcanza con completar una única DFS

Sea G un grafo no dirigido y sea T el bosque producido por DFS.

- toda arista que no pertenece a T está contenida en un ciclo;
- por lo tanto, solamente las aristas de T pueden ser puentes.

Cantidad de puentes

Como todo puente pertenece al árbol DFS, $\# \text{puentes} \leq n - 1$.

Idea del algoritmo

- 1 Durante una única DFS, calcular para cada vértice su tiempo de descubrimiento d , su predecesor π y un valor low (*low*).
- 2 Recorrer las aristas del bosque y decidir cuáles son puentes comparando $\text{low}[v]$ con $d[\pi[v]]$.

El algoritmo también funciona si G no es conexo: en ese caso, DFS produce un bosque en lugar de un único árbol.

$d[u]$ conserva el significado de las filminas de DFS

Cuando DFS descubre un vértice u , incrementa el contador global *tiempo* y asigna

$$d[u] \leftarrow \text{tiempo}.$$

De este modo, al finalizar,

$$d : V(G) \longrightarrow \{1, \dots, 2|V(G)|\}$$

es la función que asigna a cada vértice su tiempo de descubrimiento.

Además, si u fue descubierto al examinar la arista $\{\pi[u], u\}$, entonces $\pi[u]$ es su padre en el bosque DFS.

Propiedad que vamos a usar

Si x es un ancestro propio de u en el bosque DFS, entonces $d[x] < d[u]$.

Para cada raíz r del bosque,

$$\pi[r] = \text{NIL}.$$

$\text{low}[u]$ indica hasta dónde puede volver el subárbol de u

Definimos $\text{low}[u]$ como el menor tiempo de descubrimiento de un vértice al que se puede llegar desde u :

- bajando cero o más aristas del árbol DFS; y luego
- usando, a lo sumo, una arista que no pertenece al árbol.

Más precisamente,

$$\text{low}[u] = \min \left\{ \begin{array}{l} d[u], \\ d[v] : \{u, v\} \text{ es una arista de retroceso y } v \text{ es ancestro de } u, \\ \text{low}[w] : \pi[w] = u \end{array} \right\}.$$

Para hallar los puentes alcanza con completar una única DFS

Sea G un grafo no dirigido y sea T el bosque producido por DFS.

Toda arista que no pertenece a T está contenida en un ciclo; por lo tanto, solamente las aristas de T pueden ser puentes.

Idea del algoritmo

- 1 Ejecutar una DFS ordinaria y calcular T , d , f y π .
- 2 Una vez terminada la DFS, calcular los valores low recorriendo T desde las hojas hacia las raíces.
- 3 Para cada arista $\{\pi[v], v\}$ del bosque, decidir si es puente comparando $\text{low}[v]$ con $d[\pi[v]]$.

low puede calcularse después de terminar DFS

Una vez conocidos el bosque DFS T , los tiempos d y los predecesores π , ejecutamos:

Procedimiento CALCULAR-LOW-OFFLINE(G, T, d, π)

para cada $u \in V(G)$ **hacer**

$\text{low}[u] \leftarrow d[u]$

para cada $\{u, v\} \in E(G) \setminus E(T)$, con v ancestro de u , **hacer**

$\text{low}[u] \leftarrow \min\{\text{low}[u], d[v]\}$

para cada $u \in V(G)$ **en postorden de** T **hacer**

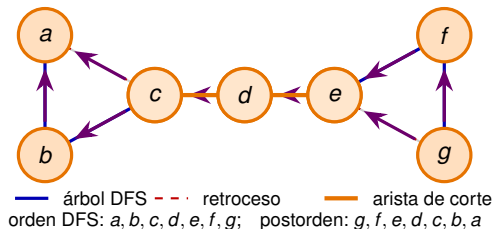
si $\pi[u] \neq \text{NIL}$ **entonces**

$\text{low}[\pi[u]] \leftarrow \min\{\text{low}[\pi[u]], \text{low}[u]\}$

retornar low

El postorden garantiza que, cuando se procesa u , los valores de todos sus hijos ya fueron calculados. La complejidad es $O(n + m)$.

Cálculo offline de low



u	$d[u]$	$f[u]$	$low[u]$
a	1	14	1
b	2	13	2 1
c	3	12	3 1
d	4	11	4
e	5	10	5
f	6	9	6 5
g	7	8	7 5

1. Inicialización.

La DFS ya terminó. Para todo vértice u ,

$$low[u] \leftarrow d[u].$$

2. Se procesa la arista de retroceso ge .

Como e es un ancestro de g ,

$$low[g] \leftarrow \min\{7, d[e]\} = 5.$$

3. Se procesa la arista de retroceso ca .

La primera fase extiende la inicialización de DFS

Procedimiento PUENTES(G)

para cada $u \in V(G)$ **hacer**

$\text{color}[u] \leftarrow \text{blanco}$

$\pi[u] \leftarrow \text{NIL}$

$d[u] \leftarrow \infty$

$\text{low}[u] \leftarrow \infty$

$\text{tiempo} \leftarrow 0$

para cada $u \in V(G)$ **hacer**

si $\text{color}[u] = \text{blanco}$ **entonces**

 DFS-PUENTES-VISIT(G, u)

Al terminar esta fase, d , low y π están definidos para todos los vértices. La segunda fase solamente inspeccionará las aristas $\{\pi[v], v\}$ del bosque DFS.

DFS-PUENTES-VISIT calcula low al retroceder

Procedimiento DFS-PUENTES-VISIT(G, u)

$tiempo \leftarrow tiempo + 1$

$d[u] \leftarrow tiempo$

$low[u] \leftarrow d[u]$

$color[u] \leftarrow \text{gris}$

para cada $v \in \text{Adj}[u]$ **hacer**

si $color[v] = \text{blanco}$ **entonces**

$\pi[v] \leftarrow u$

 DFS-PUENTES-VISIT(G, v)

$low[u] \leftarrow \min\{low[u], low[v]\}$

si no, si $color[v] = \text{gris}$ **y** $v \neq \pi[u]$ **entonces**

$low[u] \leftarrow \min\{low[u], d[v]\}$

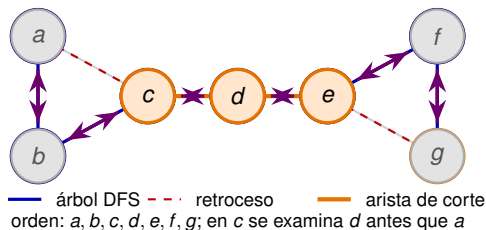
$tiempo \leftarrow tiempo + 1$

$f[u] \leftarrow tiempo$

$color[u] \leftarrow \text{negro}$

La condición $v \neq \pi[u]$ evita considerar como arista de retroceso la copia de la arista del árbol que lleva de u a su padre.

Al retroceder, DFS completa f y propaga low



u	$d[u]$	$f[u]$	$low[u]$
a	1	14	1
b	2	13	- 2 1
c	3	12	- 3 1
d	4	11	4
e	5	10	5
f	6	9	- 6 5
g	7	8	- 7 5

1. Descubre a .

$$d[a] = low[a] = 1.$$

2. Avanza por ab y descubre b .

$$d[b] = low[b] = 2.$$

3. Avanza por bc y descubre c .

$$d[c] = low[c] = 3$$

Las dos actualizaciones de low reúnen toda la información

Al descubrir u , inicialmente solamente sabemos que u se alcanza a sí mismo:

$$\text{low}[u] \leftarrow d[u].$$

Un hijo v termina de procesarse

Todo lo que puede alcanzarse desde el subárbol de v también puede alcanzarse desde el subárbol de u . Por eso,

$$\text{low}[u] \leftarrow \min\{\text{low}[u], \text{low}[v]\}.$$

Se examina una arista que no es la arista al padre

La arista uv permite alcanzar directamente un vértice ya descubierto. Por eso,

$$\text{low}[u] \leftarrow \min\{\text{low}[u], d[v]\}.$$

Como la actualización con $\text{low}[v]$ se realiza después de la llamada recursiva, los valores se propagan de las hojas hacia la raíz.

La segunda fase prueba una desigualdad por arista del árbol

Después de completar todas las llamadas a DFS-PUENTES-VISIT, el procedimiento PUENTES(G) continúa así:

$B \leftarrow \emptyset$

para cada $v \in V(G)$ **hacer**

si $\pi[v] \neq \text{NIL}$ **y** $\text{low}[v] > d[\pi[v]]$ **entonces**

$B \leftarrow B \cup \{\{\pi[v], v\}\}$

retornar B

No es necesario recorrer todo $E(G)$ en esta fase: alcanza con recorrer los vértices no raíz, uno por cada arista del bosque DFS.

Teorema

Sea G un grafo con $uv \in E(G)$ y π producido por el algoritmo DFS. Entonces, uv es puente de G , con $\pi[v] = u$ si y solo si $\text{low}[v] > d[u]$

El algoritmo encuentra todos los puentes en tiempo lineal

- La inicialización cuesta $\Theta(|V|)$.
- Cada vértice es descubierto una sola vez.
- Cada arista aparece dos veces en las listas de adyacencia y provoca solamente una cantidad constante de operaciones.
- La segunda fase recorre $V(G)$ una vez.

Por lo tanto, con listas de adyacencia,

$$\Theta(|V| + |E|).$$

Se almacenan los arreglos `color`, `$\pi$` , `$d$` , `$f$` y `low`, además de la pila de recursión y la salida. El espacio adicional es

$$O(|V|)$$

sin contar la representación del grafo ni la lista de puentes devuelta.